Delivery routing with QAOA: a fair comparison on a real map

TEAM NAME · Member 1, Member 2 · Qiskit Fall Fest Hackathon 2026, Problem 06 (QAOA & optimisation) · code: https://github.com/YOUR-USERNAME/YOUR-REPO

1. Problem

A delivery company must decide *who delivers what* and *in which order*. Both are routing problems (variants of the Travelling Salesperson Problem, TSP) whose number of possible answers grows factorially with the number of stops. We ask: can QAOA, run on a simulator and on a real IBM quantum computer, solve a small but real instance, and where does the approach stop being useful compared with classical methods? **Success** means: a correct QUBO encoding (checked against exact classical solutions), QAOA output that is measurably better than random guessing, an honest comparison with classical solvers given similar resources, and a clear statement of where the quantum approach breaks down.

2. Method

- **Map.** A depot (Sealdah) and 9 well-known places in Kolkata (approximate coordinates). For the main result we use great-circle distances, so the result is reproducible offline; the dashboard can also use real road distances from OpenStreetMap (OSRM).
- **Classical solvers** (our own code, no solver libraries): nearest-neighbour + 2-opt heuristic; Held–Karp dynamic programming (exact, $O(n^2 2^n)$); brute force (exact, $(n-1)!/2$ tours).
- **Encoding A, full route** $(n-1)^2$ qubits: $x_{i,t}$=1 if stop i is visited at step t. Energy = route length + A·(one stop per step)^2 + A·(each stop once)^2.
- **Encoding B, two vans** (main result), one qubit per stop: x_i=1 if van B serves stop i (stop 1 fixed to van A, so 8 qubits). Energy = –(Clarke–Wright savings $s_{ij} = d_{0i}+d_{0j}-d_{ij}$ of stops sharing a van) + A·$(|B| - 4)^2$ (capacity: van B takes 4 of the 9 stops). Each van is then routed exactly with Held–Karp.
- **QAOA.** QUBO → Ising ($x = (1-z)/2$); circuit = H on all qubits, then p layers of RZZ/RZ (cost) and RX (mixer). Angles γ, β tuned on our own state-vector simulator (grid search + Nelder–Mead, 183 evaluations, p = 1); only the final tuned circuit is run with 4,000 shots.
- **Three quantum back-ends, kept separate:** (i) ideal simulation (exact state vector); (ii) Qiskit Aer with the noise model of IBM's ibm_kingston (a *noise model*, not hardware; 3 seeds); (iii) a real IBM device via Qiskit Runtime SamplerV2 with dynamical decoupling and gate twirling. The compiled circuit has 92 two-qubit gates (depth 201).
- **Checks** (unit tests in tests/): QUBO energy of the optimal tour equals its length; the Qiskit circuit gives the same probabilities as our simulator; brute force equals Held–Karp; heuristic never beats the optimum; street distances reach the hardware circuit.

3. Results

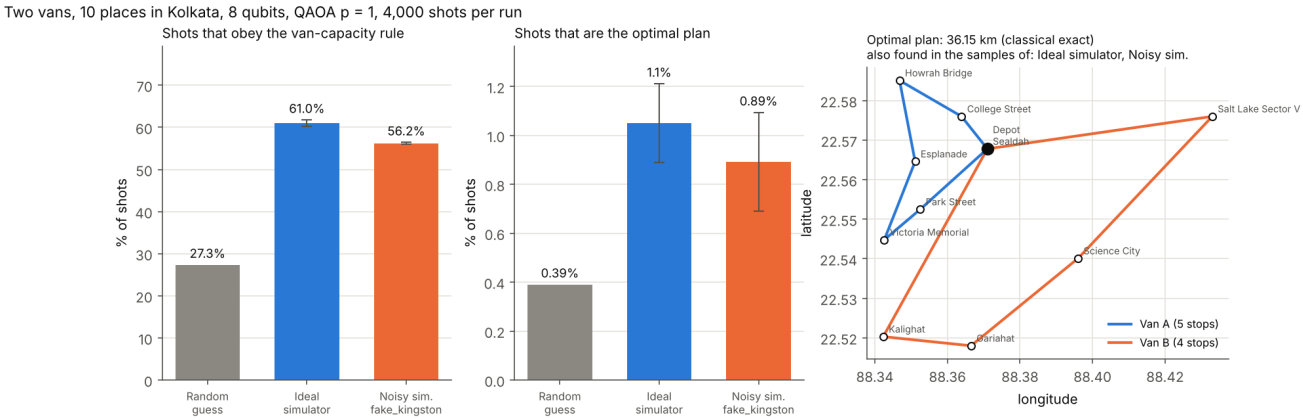


Figure 1 (main result). Share of shots that obey the capacity rule (left) and that are the optimal plan (middle), with binomial standard errors (noisy simulator: spread over 3 seeds). Right: the optimal plan.

Source	Valid shots	Optimal-plan shots	Best plan found	Gap vs optimum
Random guessing	27.3%	0.39%	—	—
Ideal simulator (exact state)	62.7%	1.00%	—	—
Ideal simulator (4,000 shots)	61.0% ± 0.8%	1.05% ± 0.16%	36.15 km	0.0%
Noisy simulator (fake_kingston, 3 seeds)	56.2% ± 0.2%	0.89% ± 0.20%	36.15 km	0.0%

Classical exact (all 70 fair splits)	—	—	36.15 km	0 (7.0 ms)
Classical sweep heuristic	—	—	36.15 km	0.0% (0.8 ms)

QAOA concentrates probability on good answers: valid splits rise from 27.3% (random) to 62.7% (ideal), and the optimal plan from 0.39% to 1.00%, about 2.6× random. The noise model costs only a few points at this circuit size. Because a few thousand shots cover all 256 bitstrings many times, every source finds the optimal 36.15 km plan, but so does the classical sweep heuristic, in under a millisecond. No real-hardware run is included in this version (see run_hardware.py).

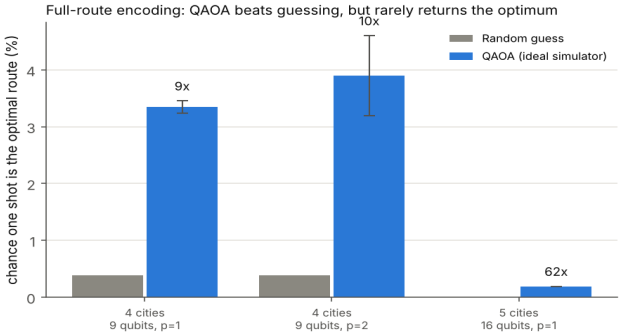


Figure 2. Full-route encoding, ideal simulator, mean ± s.d. over random maps. With 4 cities (9 qubits) one shot is optimal 3.3% of the time (9× random); with 5 cities (16 qubits) only 0.19%. Held–Karp solves both exactly in under 0.1 ms.

Classical scaling (figures/classical_scaling.png): brute force takes 1.6 s at 11 cities, Held–Karp 0.17 s at 14 cities, and the heuristic 0.20 s at 60 cities. The full-route encoding would need 59^2 = 3,481 qubits for 60 cities.

4. Honest limits

- **No quantum advantage.** Classical methods match or beat QAOA on every instance here, in less time. Held–Karp is exact, so no method can find a shorter route; at best a quantum method could match it faster, and our measurements show the opposite. TSP is NP-hard and QAOA has no proven speed-up for it.
- **Small sizes.** The full-route encoding needs $(n–1)^2$ qubits and only $(n–1)!/2$ of the $2^{(n–1)^2}$ bitstrings are valid tours (1 in 10^{19} at 10 cities). Exact simulation stops at about 5 cities (16 qubits); on hardware, noise from the 140–440 two-qubit gates limits it to about 4.
- **The two-van result is easy for sampling.** With 8 qubits, 4,000 shots almost certainly contain the optimum even for random guessing (expected ~16 hits). The meaningful metric is the *probability* of good answers, not "best plan found".
- **Proxy objective.** QAOA optimises savings within a van, not the exact route length; on this map its optimum coincides with the true best split, but that is not guaranteed in general.
- **Assumptions.** Approximate coordinates; great-circle distances in the main result (road distances in the dashboard are averaged over both directions); a fixed 5/4 capacity split; p = 1; angles tuned on a noiseless simulator and not re-tuned on hardware.
- **Not tested:** larger p, other mixers or encodings (e.g. n log n qubits), error mitigation beyond decoupling and twirling, repeated hardware runs on different days.

5. AI-use note

We used Claude (Anthropic, in the Cowork app) for code, debugging and drafting, and the IBM Quantum Platform for hardware. **One thing the AI got wrong:** its first two-van QUBO grouped stops by distance (a Max-Cut) without a capacity rule. QAOA's best plans were 15–29% longer than the best split found by our exhaustive classical check. The cause: without capacity, the cheapest plan is one van doing everything, so the proxy did not match the real goal. We added the capacity penalty and Clarke–Wright savings; the gap is now 0%. Details and a second error in AI_USE_NOTE.md.

Software: Python 3.13.16, Qiskit 2.5.2, Qiskit Aer 0.17.2, qiskit-ibm-runtime 0.50.0, NumPy 2.5.3. Generated 2026-10-13 by `python reproduce.py`.